Contents

Team Composition Skill Authoring Guide 3
1. Architecture Overview . . . 3
Three-Layer Architecture . . . 3
Execution Flow . . . 3
Key Design Principles . . . 4
2. Prerequisites . . . 4
2.1 Agent Registry Entries . . . 4
2.2 Stdin/Stdout Contract Schemas . . . 5
2.3 Team Config Template . . . 7
2.4 Inter-Wave Scripts (Optional) . . . 9
3. Step-by-Step: Creating a New Team Skill . . . 10
Step 3.1: Define the Agents . . . 10
Step 3.2: Create Stdin/Stdout Contract Schemas . . . 11
Step 3.3: Create the Team Config Template . . . 17
Step 3.4: Write the SKILL.md File . . . 19
Phase 3B: Background Team-Run (Preferred) . . . 20
Setup . . . 20
Stdin File Generation . . . 20
Launch . . . 22
Phase 3A: Foreground Fallback . . . 23
User-Facing Output . . . 23
Testing . . . 24
Dry Run (Config Generation Only) . . . 24
Manual Launch . . . 24
Verify Contracts . . . 24
Cost Verification . . . 25
Common Pitfalls . . . 25
1. Missing Tools in allowed_tools . . . 25
2. Schema Mismatch . . . 25
3. Inter-Wave File Timing . . . 25
4. Budget Underestimation . . . 26
5. Timeout Too Short . . . 26
4. Schema Resolution Deep Dive . . . 28
Resolution Algorithm . . . 28
Examples . . . 29
Naming Convention . . . 29
What Gets Embedded . . . 29
5. Budget Planning Guide . . . 30
Cost Estimation Model . . . 30
Budget Formula . . . 30
Budget Tracking Flow . . . 30
Example Walkthrough . . . 31
Budget Floor . . . 32
6. Wave Composition Patterns . . . 32
Pattern A: Single Wave (Parallel) . . . 32
Pattern B: Two Waves with Synthesis . . . 33
Pattern C: DAG-Ordered Waves . . . 34
7. Stdin File Generation . . . 36
Common Envelope (Required for ALL) . . . 36
Workflow-Specific Fields . . . 37
Generation Template (JavaScript) . . . 39
Validation . . . 39

8. Stdout Extraction Pipeline . . . 40
Tier 1: Direct JSON Parse . . . 40
Tier 2: Code Block Extraction . . . 40
Validation Rules . . . 42
Output File Structure . . . 42
9. Testing Your Skill . . . 43
9.1 Dry Run (Config Generation Only) . . . 43
9.2 Manual Launch . . . 44
9.3 Verify Stdout Contracts . . . 45
9.4 Cost Verification . . . 46
9.5 Integration Test . . . 47
10. Reference: Existing Workflows . . . 49
Overview Table . . . 49
Review Workflow . . . 49
Braintrust Workflow . . . 50
Implementation Workflow . . . 51
11. Reference: Complete File Inventory . . . 52
Go Binaries (Execution Layer) . . . 52
Schemas (Contract Layer) . . . 52
Configuration Files . . . 53
Runtime Files (Per Team Execution) . . . 53
Skills (User Interface Layer) . . . 54
Validation Tickets (Documentation) . . . 54
Architecture Documentation . . . 54
12. Checklist: New Team Skill . . . 55
Planning Phase . . . 55
Agent Registration . . . 55
Schema Creation . . . 55
Template Creation . . . 55
Skill Implementation . . . 55
Inter-Wave Script (if applicable) . . . 56
Testing . . . 56
Documentation . . . 56
Deployment . . . 56
Appendix A: Common Patterns . . . 56
Pattern: User-Scoped Configuration . . . 56
Pattern: Context Injection . . . 57
Pattern: Progressive Budget Increase . . . 57
Pattern: Conditional Wave Inclusion . . . 57
Appendix B: Debugging Techniques . . . 58
Technique 1: Verbose Runner Logging . . . 58
Technique 2: Inspect Intermediate Files . . . 58
Technique 3: Manual Agent Invocation . . . 58
Technique 4: Cost Trace . . . 59
Technique 5: Heartbeat Monitoring . . . 59
Appendix C: Performance Tuning . . . 59
Tuning 1: Adjust Timeouts . . . 59
Tuning 2: Parallelize More Aggressively . . . 59
Tuning 3: Use Haiku for Simple Tasks . . . 60
Tuning 4: Reduce Thinking Budget . . . 60

Team Composition Skill Authoring Guide

Version: 1.0.0 **Last Updated:** 2026-02-08 **Audience:** Developers creating new workflow skills that leverage team orchestration

---

1. Architecture Overview

The GOgent-Fortress team orchestration system enables complex multi-agent workflows to run in the background while users maintain control over their terminal. This system has proven effective in production with three validated workflows: `/review`, `/braintrust`, and `/implement`.

Three-Layer Architecture

```
+-------------------------------------------------------------+
|                    LAYER 1: SKILLS                          |
|  User-facing entry points (SKILL.md files)                  |
|  - Parse user request                                       |
|  - Generate team config.json + stdin files                  |
|  - Launch background daemon                                 |
|-------------------------------------------------------------┘
                            |
                            v
+-------------------------------------------------------------+
|              LAYER 2: GO ORCHESTRATION ENGINE               |
|  Background execution (gogent-team-run)                     |
|  - Wave-by-wave agent spawning                              |
|  - Budget tracking with cost reconciliation                 |
|  - Process management (spawn, monitor, kill)                |
|  - Inter-wave synthesis (optional)                          |
|-------------------------------------------------------------┘
                            |
                            v
+-------------------------------------------------------------+
|            LAYER 3: JSON SCHEMA CONTRACTS                   |
|  Typed interfaces between components                        |
|  - Stdin schemas: What agents receive                       |
|  - Stdout schemas: What agents must produce                 |
|  - Config schema: Team state and wave definitions           |
|-------------------------------------------------------------┘
```

Execution Flow

```
User invokes skill (e.g., /security-audit)
  |
  |-► [SKILL] Read template schemas/teams/security-audit.json
  |-► [SKILL] Populate dynamic fields (session_id, timestamps, project_root)
  |-► [SKILL] Write {team_dir}/config.json
  |-► [SKILL] For each member: generate stdin_{member_id}.json from contract
  |-► [SKILL] Launch: gogent-team-run "{team_dir}" & (background)
  |-► [SKILL] Verify launch (check config.json for background_pid)
  |-► [SKILL] Return to user with monitoring instructions
        |
```

```
      v
gogent-team-run daemon (background)
  |
  |-► Wave 1: Spawn agents in parallel
  |     |-► For each member: tryReserveBudget(estimated)
  |     |-► Spawn: claude --agent {id} < stdin_{id}.json > stdout_{id}.json
  |     |-► Monitor: Wait for all processes in wave to complete
  |     |-► Extract: Parse stdout files, reconcileCost(estimated, actual)
  |
  |-► (Optional) Run on_complete_script between waves
  |     |-► Example: gogent-team-prepare-synthesis merges Wave 1 outputs
  |
  |-► Wave 2: Spawn next wave (may read artifacts from Wave 1)
  |     |-► Repeat spawn -> monitor -> extract cycle
  |
  |-► Complete: Write final costs, update config.json status
        |
        v
User checks results
  |-► /team-status -> Shows real-time progress
  |-► /team-result -> Displays stdout files when complete
  |-► /team-cancel -> Graceful shutdown if needed
```

Key Design Principles

1. **Orchestrators don't touch implementation files**: They coordinate via JSON output, file I/O happens in Go native code outside Claude's tool permissions. This allows orchestrators to run as background processes without interactive tool approval.
2. **Cost tracking has guardrails**: Pre-spawn budget reservation prevents overruns. Post-spawn reconciliation adjusts for estimation error. Budget never goes negative (clamped to $0.00).
3. **Graceful degradation**: Three-tier stdout extraction ensures system continues even if agents don't follow schema perfectly. Contract compliance is validated but non-blocking.
4. **File-based IPC**: All communication between user, skill, daemon, and agents happens via filesystem. No pipes, no sockets. Enables robust process isolation and monitoring.

---

2. Prerequisites

Before creating a new team skill, ensure these components exist:

2.1 Agent Registry Entries

All agents your workflow will use must be registered in `~/.claude/agents/agents-index.json`. Each agent needs:

```
{
  "id": "agent-unique-id",
  "model": "haiku" | "sonnet" | "opus",
  "tier": 1 | 2 | 3,
  "description": "What this agent does",
  "cli_flags": {
    "allowed_tools": ["Read", "Glob", "Grep", "Bash"],
```

```
    "permission_mode": "delegate"
  },
  "spawned_by": ["router", "orchestrator-id"],
  "can_spawn": ["child-agent-id"]
}
```

Critical fields:

- `cli_flags.allowed_tools`: Determines what file operations the agent can perform. For implementation workers that create files, `augmentToolsForImplementation()` in `spawn.go` automatically adds `Write` and `Edit` when `workflowType == "implementation"`. For other workflows, explicitly include needed tools.
- `model`: Drives cost estimation and timeout defaults (haiku: 120s, sonnet: 300s, opus: 600s)

2.2 Stdin/Stdout Contract Schemas

Location: `~/.claude/schemas/teams/stdin-stdout/{workflow}-{agent}.json`

These define the typed interface between your skill and agents. Structure:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Security Audit Vulnerability Scanner Contract",
  "description": "Stdin and stdout contract for vulnerability-scanner agent in security-audit w
  "version": "1.0.0",

  "stdin": {
    "type": "object",
    "required": ["agent", "workflow", "context", "description"],
    "properties": {
      "agent": {
        "type": "string",
        "enum": ["vulnerability-scanner"]
      },
      "workflow": {
        "type": "string",
        "enum": ["security-audit"]
      },
      "context": {
        "type": "object",
        "required": ["project_root", "team_dir"],
        "properties": {
          "project_root": {"type": "string"},
          "team_dir": {"type": "string"}
        }
      },
      "description": {
        "type": "string",
        "description": "Human-readable task description"
      },
      "scan_scope": {
        "type": "object",
        "description": "Workflow-specific: what to scan",
        "properties": {
```

```
          "file_patterns": {
            "type": "array",
            "items": {"type": "string"}
          },
          "exclude_patterns": {
            "type": "array",
            "items": {"type": "string"}
          }
        }
      }
    }
  },

  "stdout": {
    "type": "object",
    "required": ["$schema", "status", "metadata"],
    "properties": {
      "$schema": {
        "type": "string",
        "enum": ["security-audit-vulnerability-scanner"],
        "description": "Must match filename without .json extension"
      },
      "status": {
        "type": "string",
        "enum": ["complete", "partial", "failed"]
      },
      "metadata": {
        "type": "object",
        "required": ["thinking_budget_used"],
        "properties": {
          "thinking_budget_used": {"type": "integer"}
        }
      },
      "findings": {
        "type": "array",
        "description": "Workflow-specific: vulnerability reports",
        "items": {
          "type": "object",
          "properties": {
            "severity": {"type": "string", "enum": ["critical", "high", "medium", "low"]},
            "file": {"type": "string"},
            "line": {"type": "integer"},
            "description": {"type": "string"},
            "remediation": {"type": "string"}
          }
        }
      }
    }
  }
}
```

Common stdin fields (required for ALL workflows): - `agent`: Agent ID from agents-index.json - `workflow`: Workflow type string (must match across all files) - `context.project_root`: Absolute path to project root - `context.team_dir`: Absolute path to team execution directory - `description`: Human-readable

task summary

Common stdout fields (required for ALL workflows): - `$schema`: Must match filename without `.json` (e.g., `security-audit-vulnerability-scanner`) - `status`: One of `complete`, `partial`, `failed` - `metadata.thinking_budget_used`: Integer token count

Workflow-specific fields: Add whatever your agents need to do their job.

2.3 Team Config Template

Location: `~/.claude/schemas/teams/{workflow}.json`

This is the master template for your workflow. It defines waves, members, budget, and dependencies.

```
{
  "$schema": "./team-config.json",
  "version": "1.0.0",
  "team_name": "security-audit-TIMESTAMP",
  "workflow_type": "security-audit",
  "project_root": "",
  "session_id": "",
  "created_at": "",
  "budget_max_usd": 3.0,
  "budget_remaining_usd": 3.0,
  "warning_threshold_usd": 2.4,
  "status": "pending",
  "background_pid": null,
  "waves": [
    {
      "wave_number": 1,
      "description": "Parallel security scanning",
      "status": "pending",
      "started_at": null,
      "completed_at": null,
      "on_complete_script": "gogent-team-prepare-synthesis",
      "members": [
        {
          "member_id": "vulnerability-scanner",
          "agent": "vulnerability-scanner",
          "model": "haiku",
          "description": "Scan for common vulnerability patterns",
          "status": "pending",
          "process_pid": null,
          "exit_code": null,
          "cost_estimated_usd": 0.1,
          "cost_actual_usd": 0.0,
          "cost_status": "",
          "error_message": "",
          "retry_count": 0,
          "max_retries": 2,
          "timeout_ms": 120000,
          "started_at": null,
          "completed_at": null,
          "blocked_by": [],
          "blocks": []
        },
```

```
      {
        "member_id": "dependency-checker",
        "agent": "dependency-checker",
        "model": "haiku",
        "description": "Check for vulnerable dependencies",
        "status": "pending",
        "process_pid": null,
        "exit_code": null,
        "cost_estimated_usd": 0.1,
        "cost_actual_usd": 0.0,
        "cost_status": "",
        "error_message": "",
        "retry_count": 0,
        "max_retries": 2,
        "timeout_ms": 120000,
        "started_at": null,
        "completed_at": null,
        "blocked_by": [],
        "blocks": []
      },
      {
        "member_id": "secret-detector",
        "agent": "secret-detector",
        "model": "haiku",
        "description": "Detect hardcoded secrets and credentials",
        "status": "pending",
        "process_pid": null,
        "exit_code": null,
        "cost_estimated_usd": 0.1,
        "cost_actual_usd": 0.0,
        "cost_status": "",
        "error_message": "",
        "retry_count": 0,
        "max_retries": 2,
        "timeout_ms": 120000,
        "started_at": null,
        "completed_at": null,
        "blocked_by": [],
        "blocks": []
      }
    ]
  },
  {
    "wave_number": 2,
    "description": "Findings synthesis",
    "status": "pending",
    "started_at": null,
    "completed_at": null,
    "on_complete_script": null,
    "members": [
      {
        "member_id": "security-analyst",
        "agent": "security-analyst",
```

```
          "model": "sonnet",
          "description": "Synthesize findings into prioritized report",
          "status": "pending",
          "process_pid": null,
          "exit_code": null,
          "cost_estimated_usd": 1.5,
          "cost_actual_usd": 0.0,
          "cost_status": "",
          "error_message": "",
          "retry_count": 0,
          "max_retries": 2,
          "timeout_ms": 300000,
          "started_at": null,
          "completed_at": null,
          "blocked_by": [],
          "blocks": []
        }
      ]
    }
  ]
}
```

Member field reference:

Field	Type	Default	Description
`member_id`	string	(required)	Unique identifier within team
`agent`	string	(required)	Agent ID from agents-index.json
`model`	string	(required)	haiku / sonnet / opus
`description`	string	(required)	Human-readable task
`status`	string	“pending”	pending / running / complete / failed / skipped
`process_pid`	int / null	null	OS process ID when running
`exit_code`	int / null	null	Process exit code after completion
`cost_estimated_usd`	float	(required)	Pre-spawn estimate
`cost_actual_usd`	float	0.0	Reconciled after completion
`cost_status`	string	“”	“over_budget” / “near_warning” / “”
`error_message`	string	“”	Failure details if status=failed
`retry_count`	int	0	Current retry attempt
`max_retries`	int	2	Maximum retry attempts
`timeout_ms`	int	120000 (haiku)300000 (sonnet)600000 (opus)	Execution timeout
`started_at`	string / null	null	ISO 8601 timestamp
`completed_at`	string / null	null	ISO 8601 timestamp
`blocked_by`	string[]	[]	Member IDs that must complete first
`blocks`	string[]	[]	Member IDs that wait on this

2.4 Inter-Wave Scripts (Optional)

If your workflow has multiple waves where Wave 2+ needs synthesized output from Wave 1, you can:

Option A: Reuse existing synthesis script

Use `gogent-team-prepare-synthesis` (most common). This script: - Reads all stdout files from completed wave - Merges them into `pre-synthesis.md` in team directory - Wave 2 agent reads this file via stdin reference: `pre_synthesis_path`

Option B: Write custom inter-wave script

Create a Go binary in `cmd/gogent-{workflow}-prepare/` that: - Takes team directory as first argument: `./gogent-custom-prepare /path/to/team-dir` - Reads stdout files from previous wave - Writes artifact(s) for next wave - Exits 0 on success, non-zero on failure

Place the binary name in `waves[N].on_complete_script` field in your template.

---

3. Step-by-Step: Creating a New Team Skill

We'll create a complete `/security-audit` skill that demonstrates all key patterns.

Step 3.1: Define the Agents

Add to `~/.claude/agents/agents-index.json`:

```
{
  "agents": [
    {
      "id": "vulnerability-scanner",
      "model": "haiku",
      "tier": 1,
      "description": "Scans codebase for common vulnerability patterns (SQL injection, XSS, etc
      "cli_flags": {
        "allowed_tools": ["Read", "Glob", "Grep", "Bash"],
        "permission_mode": "delegate"
      },
      "spawned_by": ["router", "security-audit-skill"],
      "can_spawn": []
    },
    {
      "id": "dependency-checker",
      "model": "haiku",
      "tier": 1,
      "description": "Checks dependencies for known CVEs using package manager audit tools",
      "cli_flags": {
        "allowed_tools": ["Read", "Bash"],
        "permission_mode": "delegate"
      },
      "spawned_by": ["router", "security-audit-skill"],
      "can_spawn": []
    },
    {
      "id": "secret-detector",
      "model": "haiku",
      "tier": 1,
      "description": "Detects hardcoded secrets, API keys, passwords in source code",
      "cli_flags": {
        "allowed_tools": ["Read", "Glob", "Grep"],
        "permission_mode": "delegate"
```

```
      },
      "spawned_by": ["router", "security-audit-skill"],
      "can_spawn": []
    },
    {
      "id": "security-analyst",
      "model": "sonnet",
      "tier": 2,
      "description": "Synthesizes security findings into prioritized remediation report",
      "cli_flags": {
        "allowed_tools": ["Read", "Write"],
        "permission_mode": "delegate"
      },
      "spawned_by": ["router", "security-audit-skill"],
      "can_spawn": []
    }
  ]
}
```

Note: Wave 1 agents use `Read`, `Glob`, `Grep`, `Bash` for scanning. Wave 2 analyst uses `Read` (to read Wave 1 outputs) and `Write` (to generate final report).

Step 3.2: Create Stdin/Stdout Contract Schemas

Create four contract files in `~/.claude/schemas/teams/stdin-stdout/`:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Security Audit Vulnerability Scanner Contract",
  "version": "1.0.0",

  "stdin": {
    "type": "object",
    "required": ["agent", "workflow", "context", "description", "scan_scope"],
    "properties": {
      "agent": {"type": "string", "enum": ["vulnerability-scanner"]},
      "workflow": {"type": "string", "enum": ["security-audit"]},
      "context": {
        "type": "object",
        "required": ["project_root", "team_dir"],
        "properties": {
          "project_root": {"type": "string"},
          "team_dir": {"type": "string"}
        }
      },
      "description": {"type": "string"},
      "scan_scope": {
        "type": "object",
        "properties": {
          "file_patterns": {"type": "array", "items": {"type": "string"}},
          "exclude_patterns": {"type": "array", "items": {"type": "string"}},
          "vulnerability_types": {
```

```
            "type": "array",
            "items": {"type": "string"},
            "description": "Types to check: sql_injection, xss, path_traversal, etc."
          }
        }
      }
    }
  },

  "stdout": {
    "type": "object",
    "required": ["$schema", "status", "metadata", "findings"],
    "properties": {
      "$schema": {"type": "string", "enum": ["security-audit-vulnerability-scanner"]},
      "status": {"type": "string", "enum": ["complete", "partial", "failed"]},
      "metadata": {
        "type": "object",
        "required": ["thinking_budget_used", "files_scanned"],
        "properties": {
          "thinking_budget_used": {"type": "integer"},
          "files_scanned": {"type": "integer"},
          "patterns_checked": {"type": "integer"}
        }
      },
      "findings": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["severity", "type", "file", "line", "description"],
          "properties": {
            "severity": {"type": "string", "enum": ["critical", "high", "medium", "low", "info
            "type": {"type": "string"},
            "file": {"type": "string"},
            "line": {"type": "integer"},
            "description": {"type": "string"},
            "code_snippet": {"type": "string"},
            "remediation": {"type": "string"}
          }
        }
      },
      "summary": {
        "type": "object",
        "properties": {
          "critical_count": {"type": "integer"},
          "high_count": {"type": "integer"},
          "medium_count": {"type": "integer"},
          "low_count": {"type": "integer"}
        }
      }
    }
  }
}
```

security-audit-vulnerability-scanner.json

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Security Audit Dependency Checker Contract",
  "version": "1.0.0",

  "stdin": {
    "type": "object",
    "required": ["agent", "workflow", "context", "description"],
    "properties": {
      "agent": {"type": "string", "enum": ["dependency-checker"]},
      "workflow": {"type": "string", "enum": ["security-audit"]},
      "context": {
        "type": "object",
        "required": ["project_root", "team_dir"],
        "properties": {
          "project_root": {"type": "string"},
          "team_dir": {"type": "string"}
        }
      },
      "description": {"type": "string"},
      "package_managers": {
        "type": "array",
        "items": {"type": "string"},
        "description": "npm, go, pip, etc."
      }
    }
  },

  "stdout": {
    "type": "object",
    "required": ["$schema", "status", "metadata", "vulnerabilities"],
    "properties": {
      "$schema": {"type": "string", "enum": ["security-audit-dependency-checker"]},
      "status": {"type": "string", "enum": ["complete", "partial", "failed"]},
      "metadata": {
        "type": "object",
        "required": ["thinking_budget_used"],
        "properties": {
          "thinking_budget_used": {"type": "integer"},
          "dependencies_checked": {"type": "integer"}
        }
      },
      "vulnerabilities": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["severity", "package", "cve", "description"],
          "properties": {
            "severity": {"type": "string", "enum": ["critical", "high", "medium", "low"]},
            "package": {"type": "string"},
            "version": {"type": "string"},
            "cve": {"type": "string"},
```

```
            "description": {"type": "string"},
            "fixed_in": {"type": "string"},
            "remediation": {"type": "string"}
          }
        }
      }
    }
  }
}
```

security-audit-dependency-checker.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Security Audit Secret Detector Contract",
  "version": "1.0.0",

  "stdin": {
    "type": "object",
    "required": ["agent", "workflow", "context", "description"],
    "properties": {
      "agent": {"type": "string", "enum": ["secret-detector"]},
      "workflow": {"type": "string", "enum": ["security-audit"]},
      "context": {
        "type": "object",
        "required": ["project_root", "team_dir"],
        "properties": {
          "project_root": {"type": "string"},
          "team_dir": {"type": "string"}
        }
      },
      "description": {"type": "string"},
      "secret_patterns": {
        "type": "array",
        "items": {"type": "string"},
        "description": "Types to detect: api_key, password, token, etc."
      }
    }
  },

  "stdout": {
    "type": "object",
    "required": ["$schema", "status", "metadata", "secrets"],
    "properties": {
      "$schema": {"type": "string", "enum": ["security-audit-secret-detector"]},
      "status": {"type": "string", "enum": ["complete", "partial", "failed"]},
      "metadata": {
        "type": "object",
        "required": ["thinking_budget_used"],
        "properties": {
          "thinking_budget_used": {"type": "integer"},
          "files_scanned": {"type": "integer"}
```

```
        }
      },
      "secrets": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["severity", "type", "file", "line"],
          "properties": {
            "severity": {"type": "string", "enum": ["critical", "high", "medium", "low"]},
            "type": {"type": "string"},
            "file": {"type": "string"},
            "line": {"type": "integer"},
            "description": {"type": "string"},
            "masked_value": {"type": "string"},
            "remediation": {"type": "string"}
          }
        }
      }
    }
  }
}
```

security-audit-secret-detector.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Security Audit Analyst Contract",
  "version": "1.0.0",

  "stdin": {
    "type": "object",
    "required": ["agent", "workflow", "context", "description", "pre_synthesis_path"],
    "properties": {
      "agent": {"type": "string", "enum": ["security-analyst"]},
      "workflow": {"type": "string", "enum": ["security-audit"]},
      "context": {
        "type": "object",
        "required": ["project_root", "team_dir"],
        "properties": {
          "project_root": {"type": "string"},
          "team_dir": {"type": "string"}
        }
      },
      "description": {"type": "string"},
      "pre_synthesis_path": {
        "type": "string",
        "description": "Path to merged findings from Wave 1 (created by inter-wave script)"
      },
      "report_format": {
        "type": "string",
        "enum": ["markdown", "json", "html"],
        "default": "markdown"
```

```
      }
    }
  },

  "stdout": {
    "type": "object",
    "required": ["$schema", "status", "metadata", "report"],
    "properties": {
      "$schema": {"type": "string", "enum": ["security-audit-analyst"]},
      "status": {"type": "string", "enum": ["complete", "partial", "failed"]},
      "metadata": {
        "type": "object",
        "required": ["thinking_budget_used"],
        "properties": {
          "thinking_budget_used": {"type": "integer"},
          "total_findings": {"type": "integer"},
          "critical_count": {"type": "integer"},
          "high_count": {"type": "integer"}
        }
      },
      "report": {
        "type": "object",
        "required": ["summary", "prioritized_findings"],
        "properties": {
          "summary": {"type": "string"},
          "prioritized_findings": {
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "rank": {"type": "integer"},
                "severity": {"type": "string"},
                "category": {"type": "string"},
                "description": {"type": "string"},
                "impact": {"type": "string"},
                "remediation": {"type": "string"},
                "estimated_effort": {"type": "string"}
              }
            }
          },
          "report_path": {
            "type": "string",
            "description": "Path to full report file written by analyst"
          }
        }
      }
    }
  }
}
```

security-audit-analyst.json **CRITICAL PATTERN:** Notice `security-analyst` has `pre_synthesis_path` in stdin. This file doesn't exist when you write the stdin file. It's created by the inter-wave script (`gogent-team-prepare-synthesis`) after Wave 1 completes. The analyst reads it at runtime.

Step 3.3: Create the Team Config Template

Create `~/.claude/schemas/teams/security-audit.json`:

```
{
  "$schema": "./team-config.json",
  "version": "1.0.0",
  "team_name": "security-audit-TIMESTAMP",
  "workflow_type": "security-audit",
  "project_root": "",
  "session_id": "",
  "created_at": "",
  "budget_max_usd": 3.0,
  "budget_remaining_usd": 3.0,
  "warning_threshold_usd": 2.4,
  "status": "pending",
  "background_pid": null,
  "waves": [
    {
      "wave_number": 1,
      "description": "Parallel security scanning (vulnerabilities, dependencies, secrets)",
      "status": "pending",
      "started_at": null,
      "completed_at": null,
      "on_complete_script": "gogent-team-prepare-synthesis",
      "members": [
        {
          "member_id": "vulnerability-scanner",
          "agent": "vulnerability-scanner",
          "model": "haiku",
          "description": "Scan for common vulnerability patterns",
          "status": "pending",
          "process_pid": null,
          "exit_code": null,
          "cost_estimated_usd": 0.1,
          "cost_actual_usd": 0.0,
          "cost_status": "",
          "error_message": "",
          "retry_count": 0,
          "max_retries": 2,
          "timeout_ms": 120000,
          "started_at": null,
          "completed_at": null,
          "blocked_by": [],
          "blocks": []
        },
        {
          "member_id": "dependency-checker",
          "agent": "dependency-checker",
          "model": "haiku",
          "description": "Check for vulnerable dependencies",
          "status": "pending",
          "process_pid": null,
          "exit_code": null,
          "cost_estimated_usd": 0.1,
```

```
        "cost_actual_usd": 0.0,
        "cost_status": "",
        "error_message": "",
        "retry_count": 0,
        "max_retries": 2,
        "timeout_ms": 120000,
        "started_at": null,
        "completed_at": null,
        "blocked_by": [],
        "blocks": []
      },
      {
        "member_id": "secret-detector",
        "agent": "secret-detector",
        "model": "haiku",
        "description": "Detect hardcoded secrets",
        "status": "pending",
        "process_pid": null,
        "exit_code": null,
        "cost_estimated_usd": 0.1,
        "cost_actual_usd": 0.0,
        "cost_status": "",
        "error_message": "",
        "retry_count": 0,
        "max_retries": 2,
        "timeout_ms": 120000,
        "started_at": null,
        "completed_at": null,
        "blocked_by": [],
        "blocks": []
      }
    ]
  },
  {
    "wave_number": 2,
    "description": "Findings synthesis and prioritization",
    "status": "pending",
    "started_at": null,
    "completed_at": null,
    "on_complete_script": null,
    "members": [
      {
        "member_id": "security-analyst",
        "agent": "security-analyst",
        "model": "sonnet",
        "description": "Synthesize findings into prioritized report",
        "status": "pending",
        "process_pid": null,
        "exit_code": null,
        "cost_estimated_usd": 1.5,
        "cost_actual_usd": 0.0,
        "cost_status": "",
        "error_message": "",
```

```
          "retry_count": 0,
          "max_retries": 2,
          "timeout_ms": 300000,
          "started_at": null,
          "completed_at": null,
          "blocked_by": [],
          "blocks": []
        }
      ]
    }
  ]
}
```

Budget calculation: - 3 haiku agents: 3 × $0.10 = $0.30 - 1 sonnet agent: 1 × $1.50 = $1.50 - Total estimate: $1.80 - With 50% safety margin: $1.80 × 1.5 = $2.70 - Rounded up: $3.00 - Warning threshold (80%): $2.40

Step 3.4: Write the SKILL.md File

Create `~/.claude/skills/security-audit/SKILL.md`:

```
---
name: security-audit
description: >
  Automated security audit of codebase. Spawns parallel scanners for
  vulnerabilities, dependencies, and secrets. Synthesizes findings
  into prioritized remediation report.
version: 1.0.0
triggers:
  - security audit
  - audit security
  - vulnerability scan
  - security review
examples:
  - /security-audit
  - /security-audit --scope backend
  - /security-audit --exclude tests
---

# Security Audit Skill

Performs comprehensive security audit with parallel scanning and synthesis.

## Architecture

**Wave 1** (Parallel, ~30 seconds):
- `vulnerability-scanner` (haiku): Code pattern analysis
- `dependency-checker` (haiku): CVE database checks
- `secret-detector` (haiku): Hardcoded secret detection

**Inter-Wave**: `gogent-team-prepare-synthesis` merges findings

**Wave 2** (~60 seconds):
- `security-analyst` (sonnet): Prioritized remediation report
```

```
**Total runtime**: ~90 seconds
**Estimated cost**: $1.80 (actual: typically $0.50-1.20)

## Dispatch Logic

```javascript
// Check if team-run infrastructure is available
const useTeamPattern = settings.use_team_pattern &&
                       binaryExists("gogent-team-run") &&
                       binaryExists("gogent-team-prepare-synthesis");

if (useTeamPattern) {
  // Phase 3B: Background orchestration
  executeBackgroundTeam();
} else {
  // Phase 3A: Foreground fallback (Task-based)
  executeForegroundSequence();
}
```
```

Phase 3B: Background Team-Run (Preferred)

Setup

```
// 1. Create team directory
const timestamp = Date.now();
const sessionDir = process.env.GOGENT_SESSION_DIR || `~/.cache/gogent/sessions/${sessionId}`;
const teamDir = `${sessionDir}/teams/${timestamp}.security-audit`;
fs.mkdirSync(teamDir, { recursive: true });

// 2. Load and populate template
const template = readJSON("~/.claude/schemas/teams/security-audit.json");
const projectRoot = process.cwd();

template.team_name = `security-audit-${timestamp}`;
template.project_root = projectRoot;
template.session_id = sessionId;
template.created_at = new Date().toISOString();

// Parse user flags if provided
const scope = args.includes("--scope") ? args[args.indexOf("--scope") + 1] : "all";
const excludePatterns = args.includes("--exclude") ? args[args.indexOf("--exclude") + 1].split
```

Stdin File Generation

```
// 3. Generate stdin files for Wave 1 members

// stdin_vulnerability-scanner.json
const vulnerabilityScannerStdin = {
  agent: "vulnerability-scanner",
  workflow: "security-audit",
  description: "Scan codebase for common vulnerability patterns",
  context: {
```

```
    project_root: projectRoot,
    team_dir: teamDir
  },
  scan_scope: {
    file_patterns: scope === "backend" ? ["**/*.go", "**/*.py", "**/*.js"] : ["**/*"],
    exclude_patterns: excludePatterns.concat(["**/node_modules/**", "**/.git/**"]),
    vulnerability_types: ["sql_injection", "xss", "path_traversal", "command_injection", "xxe"
  }
};
fs.writeFileSync(
  `${teamDir}/stdin_vulnerability-scanner.json`,
  JSON.stringify(vulnerabilityScannerStdin, null, 2)
);

// stdin_dependency-checker.json
const dependencyCheckerStdin = {
  agent: "dependency-checker",
  workflow: "security-audit",
  description: "Check dependencies for known CVEs",
  context: {
    project_root: projectRoot,
    team_dir: teamDir
  },
  package_managers: ["npm", "go", "pip"]  // Auto-detect based on project
};
fs.writeFileSync(
  `${teamDir}/stdin_dependency-checker.json`,
  JSON.stringify(dependencyCheckerStdin, null, 2)
);

// stdin_secret-detector.json
const secretDetectorStdin = {
  agent: "secret-detector",
  workflow: "security-audit",
  description: "Detect hardcoded secrets and credentials",
  context: {
    project_root: projectRoot,
    team_dir: teamDir
  },
  secret_patterns: ["api_key", "password", "token", "secret", "credential"]
};
fs.writeFileSync(
  `${teamDir}/stdin_secret-detector.json`,
  JSON.stringify(secretDetectorStdin, null, 2)
);

// 4. Generate stdin file for Wave 2 analyst
const analystStdin = {
  agent: "security-analyst",
  workflow: "security-audit",
  description: "Synthesize security findings into prioritized report",
  context: {
    project_root: projectRoot,
```

```
    team_dir: teamDir
  },
  pre_synthesis_path: `${teamDir}/pre-synthesis.md`,  // Created by inter-wave script
  report_format: "markdown"
};
fs.writeFileSync(
  `${teamDir}/stdin_security-analyst.json`,
  JSON.stringify(analystStdin, null, 2)
);

// 5. Write final config
fs.writeFileSync(
  `${teamDir}/config.json`,
  JSON.stringify(template, null, 2)
);
```

Launch

```
// 6. Launch team-run in background
const { spawn } = require("child_process");
const teamRunProc = spawn("gogent-team-run", [teamDir], {
  detached: true,
  stdio: "ignore"
});
teamRunProc.unref();

// 7. Verify launch (give it 2 seconds to write background_pid)
await sleep(2000);
const updatedConfig = readJSON(`${teamDir}/config.json`);

if (!updatedConfig.background_pid) {
  throw new Error("Failed to launch team-run (no background_pid in config)");
}

// 8. Return to user
return `
[security-audit] Audit launched (PID ${updatedConfig.background_pid})

**Scanners** (Wave 1):
  • vulnerability-scanner — Code pattern analysis
  • dependency-checker — CVE database checks
  • secret-detector — Hardcoded secret detection

**Analyst** (Wave 2):
  • security-analyst — Findings synthesis

**Budget:** $${template.budget_max_usd.toFixed(2)}
**Team Directory:** ${teamDir}

**Monitor progress:**
  /team-status
```

```
**View results:**
  /team-result

**Cancel:**
  /team-cancel
`;
```

Phase 3A: Foreground Fallback

If team-run infrastructure is unavailable, fall back to sequential Task-based execution:

```
async function executeForegroundSequence() {
  // Wave 1: Spawn all scanners in parallel (multiple Task calls in same message)
  const task1 = Task({
    model: "haiku",
    description: "Vulnerability scanning",
    prompt: generateAgentPrompt("vulnerability-scanner", /* ... */)
  });

  const task2 = Task({
    model: "haiku",
    description: "Dependency checking",
    prompt: generateAgentPrompt("dependency-checker", /* ... */)
  });

  const task3 = Task({
    model: "haiku",
    description: "Secret detection",
    prompt: generateAgentPrompt("secret-detector", /* ... */)
  });

  // Wait for all Wave 1 tasks to complete
  // (Runtime executes them concurrently)

  // Wave 2: Synthesize findings
  const synthesis = Task({
    model: "sonnet",
    description: "Security findings synthesis",
    prompt: generateAgentPrompt("security-analyst", {
      wave1_results: [task1, task2, task3]
    })
  });

  return synthesis;
}
```

Note: Foreground mode blocks the terminal but is simpler to implement and debug.

User-Facing Output

After launching, user sees:

```
[security-audit] Audit launched (PID 42731)
```

```
**Scanners** (Wave 1):
  • vulnerability-scanner — Code pattern analysis
  • dependency-checker — CVE database checks
  • secret-detector — Hardcoded secret detection

**Analyst** (Wave 2):
  • security-analyst — Findings synthesis

**Budget:** $3.00
**Team Directory:** /home/user/.cache/gogent/sessions/abc123/teams/1707412345.security-
audit

**Monitor progress:**
  /team-status

**View results:**
  /team-result

**Cancel:**
  /team-cancel
```

Terminal returns immediately. User can continue working.

Testing

Dry Run (Config Generation Only)

Add `--dry-run` flag support to generate config without launching:

```
/security-audit --dry-run
```

Should output team directory path. Inspect files:

```
cd /path/to/team-dir
ls -la
# Expect: config.json, stdin_*.json (4 files)

cat config.json | jq '.waves[0].members | length'
# Expect: 3

cat stdin_vulnerability-scanner.json | jq '.scan_scope'
# Verify structure
```

Manual Launch

```
gogent-team-run /path/to/team-dir
```

Monitor in real-time:

```
tail -f /path/to/team-dir/runner.log
```

Verify Contracts

After completion:

```bash
# Check all stdout files have required schema field
for f in /path/to/team-dir/stdout_*.json; do
  jq '."$schema"' "$f"
done

# Validate against schema
for f in /path/to/team-dir/stdout_*.json; do
  schema=$(jq -r '."$schema"' "$f")
  ajv validate -s ~/.claude/schemas/teams/stdin-stdout/${schema}.json -d "$f"
done
```

Cost Verification

```bash
cat /path/to/team-dir/config.json | jq '
  .waves[].members[] |
  select(.status == "complete") |
  {agent, estimated: .cost_estimated_usd, actual: .cost_actual_usd}
'
```

Compare actual vs. estimated. If actual consistently exceeds estimated by >50%, update template estimates.

Common Pitfalls

1. Missing Tools in allowed_tools

Symptom: Agent writes output to stderr like "Tool 'Write' not available"

Cause: `cli_flags.allowed_tools` in agents-index.json doesn't include needed tools

Fix: Add tools to agent definition. For implementation workers that create files, ensure `Write` and `Edit` are included OR rely on `augmentToolsForImplementation()` (currently only for `workflow_type: "implementation"`).

2. Schema Mismatch

Symptom: `resolveStdoutSchema()` logs "no schema found, using fallback"

Cause: Contract filename doesn't match naming convention

Fix: Ensure contract files are named `{workflow_type}-{agent_base}.json` where `agent_base` is the agent ID with common suffixes stripped.

3. Inter-Wave File Timing

Symptom: Wave 2 agent errors with "file not found" for `pre_synthesis_path`

Cause: Agent tries to read file before inter-wave script runs

Fix: Wave 2 agent should READ the file at runtime (in its prompt execution), not expect it in stdin. The stdin field is just a path reference.

4. Budget Underestimation

Symptom: Team stops mid-execution with "budget exceeded"

Cause: Template estimates too conservative, actual costs higher

Fix: Add 50-100% safety margin. Use production-validated costs from similar workflows as baseline.

5. Timeout Too Short

Symptom: Agents killed with "timeout exceeded" before completing

Cause: Default timeout (120s haiku, 300s sonnet) insufficient for large codebases

Fix: Increase `timeout_ms` in template for members that scan large file sets.

```
### Step 3.5: Create an Inter-Wave Script (Optional)

For this workflow, we reuse the existing `gogent-team-prepare-synthesis` binary. It:

1. Reads `stdout_vulnerability-scanner.json`, `stdout_dependency-checker.json`, `stdout_secret-
detector.json`
2. Merges findings into human-readable `pre-synthesis.md`
3. Writes to team directory

The `security-analyst` agent reads this file via the `pre_synthesis_path` field in its stdin.

**If you needed a custom script**, you'd create:

`cmd/gogent-security-audit-prepare/main.go`:

```go
package main

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "sort"
)

type Finding struct {
    Severity string `json:"severity"`
    Type     string `json:"type"`
    File     string `json:"file"`
    Line     int    `json:"line"`
    Description string `json:"description"`
}

type ScannerOutput struct {
    Schema   string    `json:"$schema"`
    Status   string    `json:"status"`
    Findings []Finding `json:"findings"`
}
```
```

```
func main() {
    if len(os.Args) < 2 {
        fmt.Fprintf(os.Stderr, "Usage: %s <team-dir>\n", os.Args[0])
        os.Exit(1)
    }

    teamDir := os.Args[1]

    // Read all stdout files from Wave 1
    var allFindings []Finding

     scanners := []string{"vulnerability-scanner", "dependency-checker", "secret-
detector"}
    for _, scanner := range scanners {
       stdoutPath := filepath.Join(teamDir, fmt.Sprintf("stdout_%s.json", scanner))
        data, err := os.ReadFile(stdoutPath)
        if err != nil {
            fmt.Fprintf(os.Stderr, "Failed to read %s: %v\n", stdoutPath, err)
            continue
        }

        var output ScannerOutput
        if err := json.Unmarshal(data, &output); err != nil {
            fmt.Fprintf(os.Stderr, "Failed to parse %s: %v\n", stdoutPath, err)
            continue
        }

        allFindings = append(allFindings, output.Findings...)
    }

    // Sort by severity (critical > high > medium > low)
   severityRank := map[string]int{"critical": 0, "high": 1, "medium": 2, "low": 3, "info": 4}
    sort.Slice(allFindings, func(i, j int) bool {
      return severityRank[allFindings[i].Severity] < severityRank[allFindings[j].Severity]
    })

    // Write merged output
    synthesis := fmt.Sprintf("# Security Audit Findings\n\n")
    synthesis += fmt.Sprintf("**Total findings:** %d\n\n", len(allFindings))

    for _, f := range allFindings {
        synthesis += fmt.Sprintf("## %s: %s\n\n", f.Severity, f.Type)
        synthesis += fmt.Sprintf("**File:** %s:%d\n\n", f.File, f.Line)
        synthesis += fmt.Sprintf("%s\n\n", f.Description)
        synthesis += "---\n\n"
    }

    outPath := filepath.Join(teamDir, "pre-synthesis.md")
    if err := os.WriteFile(outPath, []byte(synthesis), 0644); err != nil {
        fmt.Fprintf(os.Stderr, "Failed to write pre-synthesis.md: %v\n", err)
        os.Exit(1)
    }

   fmt.Fprintf(os.Stderr, "Merged %d findings into %s\n", len(allFindings), outPath)
```

}

Compile and place in $PATH:

```
cd cmd/gogent-security-audit-prepare
go build -o ~/.local/bin/gogent-security-audit-prepare
```

Update template to use your custom script:

```
{
  "wave_number": 1,
  "on_complete_script": "gogent-security-audit-prepare"
}
```

---

4. Schema Resolution Deep Dive

When `gogent-team-run` spawns an agent, it needs to: 1. Determine which stdout contract to embed in the agent's prompt 2. Validate stdout against that contract after execution

The resolution algorithm uses **candidate-based lookup** with suffix stripping.

Resolution Algorithm

```go
func resolveStdoutSchema(workflowType, agentID string) (string, error) {
    schemaDir := "~/.claude/schemas/teams/stdin-stdout/"

    // Candidate 1: Exact match
    exact := fmt.Sprintf("%s-%s.json", workflowType, agentID)
    if fileExists(filepath.Join(schemaDir, exact)) {
        return exact, nil
    }

    // Candidate 2: Suffix-stripped match
    suffixes := []string{
        "-scanner", "-checker", "-detector", "-reviewer",
        "-pro", "-critical-review", "-archivist", "-writer",
        "-analyst", "-manager", "-orchestrator"
    }

    agentBase := agentID
    for _, suffix := range suffixes {
        if strings.HasSuffix(agentBase, suffix) {
            agentBase = strings.TrimSuffix(agentBase, suffix)
            break
        }
    }

    stripped := fmt.Sprintf("%s-%s.json", workflowType, agentBase)
    if fileExists(filepath.Join(schemaDir, stripped)) {
        return stripped, nil
    }

    // Candidate 3: Generic worker
```

```go
    generic := fmt.Sprintf("%s-worker.json", workflowType)
    if fileExists(filepath.Join(schemaDir, generic)) {
        return generic, nil
    }

    return "", fmt.Errorf("no schema found for workflow=%s agent=%s", workflowType, agentID)
}
```

Examples

Workflow	Agent ID	Resolution Path	Result
security-audit	vulnerability-scanner	Exact: security-audit-vulnerability-scanner.json	**Found**
security-audit	dependency-checker	Exact: security-audit-dependency-checker.json	**Found**
braintrust	staff-architect-critical-review	Exact: No -> Strip `-critical-review` -> braintrust-staff-architect.json	**Found**
implementation	go-pro	Exact: No -> Strip `-pro` -> implementation-go.json -> Not found -> Fallback: implementation-worker.json	**Found**
custom-workflow	unknown-agent	Exact: No -> No suffix to strip -> Fallback: custom-workflow-worker.json	**Found** or Error

Naming Convention

For specialized agents:

`{workflow_type}-{agent_base_name}.json`

Where `agent_base_name` is the agent ID *without* common suffixes.

For generic workers:

`{workflow_type}-worker.json`

Use this when multiple agents share the same stdout structure.

What Gets Embedded

When a schema is resolved, `gogent-team-run` injects the `stdout` section into the agent's prompt:

```
AGENT: vulnerability-scanner

[... agent instructions ...]

OUTPUT CONTRACT:
You must produce JSON matching this schema:

{
  "$schema": "security-audit-vulnerability-scanner",
  "status": "complete" | "partial" | "failed",
```

```
  "metadata": {
    "thinking_budget_used": <integer>
  },
  "findings": [ ... ]
}

CRITICAL: Include "$schema": "security-audit-vulnerability-scanner" in your output.
```

The agent sees the exact stdout structure it must produce.

---

5. Budget Planning Guide

Team execution has **pre-spawn reservation** and **post-spawn reconciliation**. Understanding this is critical for preventing overruns.

Cost Estimation Model

Based on production-validated data from TC-013abc:

Model	Estimated Cost/Agent	Actual Range (from validation)	Estimation Margin
haiku	$0.10	$0.07 - $0.12	30% over
sonnet	$1.50	$0.40 - $0.45	70% over
opus	$5.00	$0.40 - $1.13	77-92% over

Why the large margins? - Current cost estimation uses worst-case token counts - Actual agents are efficient (structured output, focused prompts) - Better to overestimate than underestimate (blocking is worse than returning budget)

Budget Formula

```
Total Estimate = sum(agent_estimated_cost_usd for each member)
Budget Max = Total Estimate × 1.5  (50% safety margin)
Warning Threshold = Budget Max × 0.8  (warn at 80%)
```

Example for security-audit:

```
Wave 1:
  vulnerability-scanner (haiku): $0.10
  dependency-checker (haiku):    $0.10
  secret-detector (haiku):       $0.10
Wave 2:
  security-analyst (sonnet):     $1.50

Total Estimate: $1.80
Budget Max:     $1.80 × 1.5 = $2.70 -> $3.00 (rounded)
Warning:        $3.00 × 0.8 = $2.40
```

Budget Tracking Flow

```
Member spawn request arrives
  |
  |-► tryReserveBudget(member.cost_estimated_usd)
```

```
|     |
|     |-► Check: budget_remaining_usd ≥ cost_estimated_usd?
|     |     YES -> Subtract from budget_remaining_usd
|     |     NO  -> Block spawn, log "insufficient budget"
|     |
|     |-► Return: allowed=true/false
|
|-► If allowed: Spawn agent process
|
|-► Wait for completion
|
|-► reconcileCost(member.cost_estimated_usd, member.cost_actual_usd)
      |
      |-► Difference = cost_actual_usd - cost_estimated_usd
      |-► Add difference back to budget_remaining_usd
      |-► Clamp: if budget_remaining_usd < 0, set to 0.0
      |
      |-► Check thresholds:
            if budget_remaining_usd < (budget_max_usd * 0.2):
              member.cost_status = "near_warning"
            if budget_remaining_usd ≤ 0:
              member.cost_status = "over_budget"
```

Example Walkthrough

Initial state:

```
{
  "budget_max_usd": 3.0,
  "budget_remaining_usd": 3.0
}
```

Wave 1 Member 1 (vulnerability-scanner):

```
Pre-spawn:
  tryReserveBudget($0.10) -> $3.00 - $0.10 = $2.90

Post-spawn:
  cost_actual_usd: $0.07
  reconcileCost($0.10, $0.07) -> difference = -$0.03
  budget_remaining_usd = $2.90 + $0.03 = $2.93
```

Wave 1 Member 2 (dependency-checker):

```
Pre-spawn:
  tryReserveBudget($0.10) -> $2.93 - $0.10 = $2.83

Post-spawn:
  cost_actual_usd: $0.08
  reconcileCost($0.10, $0.08) -> difference = -$0.02
  budget_remaining_usd = $2.83 + $0.02 = $2.85
```

Wave 1 Member 3 (secret-detector):

```
Pre-spawn:
  tryReserveBudget($0.10) -> $2.85 - $0.10 = $2.75
```

```
Post-spawn:
  cost_actual_usd: $0.06
  reconcileCost($0.10, $0.06) -> difference = -$0.04
  budget_remaining_usd = $2.75 + $0.04 = $2.79
```

Wave 2 Member 1 (security-analyst):

```
Pre-spawn:
  tryReserveBudget($1.50) -> $2.79 - $1.50 = $1.29

Post-spawn:
  cost_actual_usd: $0.42
  reconcileCost($1.50, $0.42) -> difference = -$1.08
  budget_remaining_usd = $1.29 + $1.08 = $2.37
```

Final state:

```
{
  "budget_max_usd": 3.0,
  "budget_remaining_usd": 2.37,
  "status": "complete"
}
```

Actual total cost: $0.07 + $0.08 + $0.06 + $0.42 = **$0.63** **Estimated total cost:** $1.80 **Budget returned:** $2.37 (79% of budget unused)

This shows why generous estimates are safe — the reconciliation loop returns excess.

Budget Floor

Budget never goes below $0.00. If reconciliation would make it negative, it's clamped:

```
if tc.BudgetRemainingUSD < 0 {
    tc.BudgetRemainingUSD = 0.0
}
```

This prevents displaying negative budgets in status output.

---

6. Wave Composition Patterns

Teams can have 1 to N waves. Within a wave, agents run in parallel. Waves run sequentially.

Pattern A: Single Wave (Parallel)

Used by: `/review`

```
Wave 1: [agent-A, agent-B, agent-C, agent-D]
  ↓ All spawn simultaneously
  ↓ All complete (or fail)
  ↓ No inter-wave processing
Status: complete
```

When to use: - All agents are independent - No synthesis needed - Results are self-contained

Example:

```
{
  "waves": [
    {
      "wave_number": 1,
      "description": "Parallel code review",
      "on_complete_script": null,
      "members": [
        {"agent": "backend-reviewer", ...},
        {"agent": "frontend-reviewer", ...},
        {"agent": "standards-reviewer", ...},
        {"agent": "security-reviewer", ...}
      ]
    }
  ]
}
```

Characteristics: - Fastest pattern (all parallel) - No dependencies between members - Results aggregated by user or monitoring tool

Pattern B: Two Waves with Synthesis

Used by: `/braintrust`, `/security-audit`

```
Wave 1: [agent-A, agent-B]
  ↓ Both spawn simultaneously
  ↓ Both complete
  ↓ on_complete_script: gogent-team-prepare-synthesis
  ↓ Generates: pre-synthesis.md
Wave 2: [agent-C]
  ↓ Reads pre-synthesis.md at runtime
  ↓ Synthesizes final output
Status: complete
```

When to use: - Wave 1 produces independent analyses - Wave 2 needs to synthesize/prioritize/judge - Synthesis logic is deterministic (merge, sort, format)

Example:

```
{
  "waves": [
    {
      "wave_number": 1,
      "on_complete_script": "gogent-team-prepare-synthesis",
      "members": [
        {"agent": "einstein", "model": "opus", ...},
        {"agent": "staff-architect", "model": "opus", ...}
      ]
    },
    {
      "wave_number": 2,
      "on_complete_script": null,
      "members": [
        {
          "agent": "beethoven",
          "model": "opus",
```

```
          "description": "Synthesize analyses into unified document",
          ...
        }
      ]
    }
  ]
}
```

Critical timing detail:

Wave 2 stdin references a file that doesn't exist yet:

```
{
  "agent": "beethoven",
  "pre_synthesis_path": "/path/to/team-dir/pre-synthesis.md"
}
```

This file is created by `gogent-team-prepare-synthesis` AFTER Wave 1 completes. The agent reads it at runtime using the Read tool.

Inter-wave script contract:

Input: - Team directory path as first argument - All Wave 1 stdout files exist in team directory

Output: - Write artifact(s) to team directory - Exit 0 on success, non-zero on failure - Log to stderr (captured in runner.log)

Pattern C: DAG-Ordered Waves

Used by: `/implement`

```
Wave 1: [task-001, task-002]  (no blocked_by)
  ↓ Both spawn simultaneously
Wave 2: [task-003]  (blocked_by: [task-001])
  ↓ Waits for task-001 to complete
Wave 3: [task-004]  (blocked_by: [task-002, task-003])
  ↓ Waits for both task-002 and task-003
Status: complete
```

When to use: - Tasks have dependencies (file A must exist before B uses it) - Implementation order matters - DAG structure is known upfront

Example:

```
{
  "waves": [
    {
      "wave_number": 1,
      "members": [
        {
          "member_id": "task-001",
          "agent": "go-pro",
          "description": "Implement main.go",
          "blocked_by": [],
          "blocks": ["task-003"]
        },
        {
          "member_id": "task-002",
```

```
          "agent": "go-pro",
          "description": "Implement utils.go",
          "blocked_by": [],
          "blocks": ["task-004"]
        }
      ]
    },
    {
      "wave_number": 2,
      "members": [
        {
          "member_id": "task-003",
          "agent": "go-pro",
          "description": "Implement main_test.go (depends on main.go)",
          "blocked_by": ["task-001"],
          "blocks": ["task-004"]
        }
      ]
    },
    {
      "wave_number": 3,
      "members": [
        {
          "member_id": "task-004",
          "agent": "go-pro",
          "description": "Integration test (depends on utils and main_test)",
          "blocked_by": ["task-002", "task-003"],
          "blocks": []
        }
      ]
    }
  ]
}
```

DAG computation:

For `/implement`, the `gogent-plan-impl` binary computes waves using Kahn's algorithm (topological sort):

```go
func computeWaves(tasks []Task) []Wave {
    waves := []Wave{}
    remaining := make(map[string]Task)
    inDegree := make(map[string]int)

    // Initialize in-degree counts
    for _, task := range tasks {
        remaining[task.ID] = task
        inDegree[task.ID] = len(task.BlockedBy)
    }

    waveNum := 1
    for len(remaining) > 0 {
        // Find all tasks with in-degree 0
        ready := []Task{}
        for id, task := range remaining {
```

```go
            if inDegree[id] == 0 {
                ready = append(ready, task)
            }
        }

        if len(ready) == 0 {
            // Cycle detected
            return nil
        }

        // Create wave with ready tasks
        waves = append(waves, Wave{
            Number: waveNum,
            Members: ready,
        })

        // Remove ready tasks and update in-degrees
        for _, task := range ready {
            delete(remaining, task.ID)
            for _, blockedID := range task.Blocks {
                inDegree[blockedID]--
            }
        }

        waveNum++
    }

    return waves
}
```

Failure propagation:

If a task in Wave N fails, all tasks in Wave N+1 that are blocked by it transition to `status: "skipped"`:

```go
if member.Status == "failed" {
    for _, blockedID := range member.Blocks {
        if blockedMember := findMember(blockedID); blockedMember != nil {
            blockedMember.Status = "skipped"
            blockedMember.ErrorMessage = fmt.Sprintf("Skipped due to failure in %s", member.Me
        }
    }
}
```

---

7. Stdin File Generation

Each agent in your team needs a stdin JSON file. This file provides all context the agent needs to execute its task.

Common Envelope (Required for ALL)

Every stdin file must include these fields from `common-envelope.json`:

```
{
  "agent": "{agent-id}",
  "workflow": "{workflow-type}",
  "description": "{human-readable task summary}",
  "context": {
    "project_root": "{absolute-path-to-project}",
    "team_dir": "{absolute-path-to-team-execution-dir}"
  }
}
```

Field reference:

Field	Type	Required	Description
`agent`	string	YES	Must match agent ID in agents-index.json
`workflow`	string	YES	Workflow type (security-audit, braintrust, etc.)
`description`	string	YES	Human-readable task summary (shown in logs)
`context.project_root`	string	YES	Absolute path to project root directory
`context.team_dir`	string	YES	Absolute path to team execution directory

Workflow-Specific Fields

Add whatever your agents need to do their job. Examples from production workflows:

```
{
  "agent": "backend-reviewer",
  "workflow": "review",
  "description": "Review backend code for correctness and performance",
  "context": { ... },
  "review_scope": {
    "file_patterns": ["**/*.go"],
    "exclude_patterns": ["**/*_test.go"]
  },
  "git_context": {
    "branch": "feature-x",
    "base_branch": "main",
    "changed_files": ["cmd/server/main.go", "internal/handler/auth.go"]
  },
  "focus_areas": ["error_handling", "performance", "security"],
  "project_conventions": {
    "convention_files": ["go.md", "go-api.md"]
  }
}
```

Review Workflow

```
{
  "agent": "einstein",
  "workflow": "braintrust",
  "description": "Theoretical analysis of problem",
  "context": { ... },
  "problem_brief": "Design a rate-limiting system for multi-tenant API",
  "codebase_context": {
    "project_type": "go",
    "key_files": ["cmd/server/main.go", "internal/ratelimit/limiter.go"]
  },
  "scout_findings": {
    "complexity": "medium",
    "dependencies": ["golang.org/x/time/rate"]
  },
  "analysis_axes": [
    "algorithmic_approach",
    "scalability_constraints",
    "fairness_tradeoffs"
  ]
}
```

Braintrust Workflow

```
{
  "agent": "go-pro",
  "workflow": "implementation",
  "description": "Implement rate limiter middleware",
  "context": { ... },
  "task": {
    "id": "task-001",
    "subject": "Implement rate limiter middleware",
    "description": "Create HTTP middleware that enforces per-tenant rate limits using token buc
    "acceptance_criteria": [
      "Middleware function with standard http.Handler interface",
      "Per-tenant token bucket with configurable rate and burst",
      "Returns 429 when limit exceeded with Retry-After header",
      "Unit tests with >80% coverage"
    ],
    "blocked_by": [],
    "blocks": ["task-002"]
  },
  "implementation_scope": {
    "file_to_create": "internal/middleware/ratelimit.go",
    "test_file": "internal/middleware/ratelimit_test.go"
  },
  "conventions": {
    "convention_files": ["go.md", "go-api.md"]
  }
}
```

Implementation Workflow

Generation Template (JavaScript)

```javascript
function generateStdin(agent, workflow, projectRoot, teamDir, workflowSpecificData) {
  const stdin = {
    agent: agent.id,
    workflow: workflow,
    description: agent.description,
    context: {
      project_root: projectRoot,
      team_dir: teamDir
    }
  };

  // Merge workflow-specific fields
  Object.assign(stdin, workflowSpecificData);

  return stdin;
}

// Example usage for security-audit
const stdinVulnScanner = generateStdin(
  {id: "vulnerability-scanner", description: "Scan for vulnerabilities"},
  "security-audit",
  "/home/user/project",
  "/home/user/.cache/gogent/sessions/abc/teams/123.security-audit",
  {
    scan_scope: {
      file_patterns: ["**/*.go"],
      exclude_patterns: ["**/vendor/**"],
      vulnerability_types: ["sql_injection", "xss"]
    }
  }
);

fs.writeFileSync(
  path.join(teamDir, "stdin_vulnerability-scanner.json"),
  JSON.stringify(stdinVulnScanner, null, 2)
);
```

Validation

Before writing stdin files, validate required fields:

```javascript
function validateStdin(stdin) {
  const required = ["agent", "workflow", "description", "context"];
  const contextRequired = ["project_root", "team_dir"];

  for (const field of required) {
    if (!stdin[field]) {
      throw new Error(`Missing required field: ${field}`);
    }
  }

  for (const field of contextRequired) {
```

```
    if (!stdin.context[field]) {
      throw new Error(`Missing required context field: ${field}`);
    }
  }

  // Workflow-specific validation
  if (stdin.workflow === "security-audit" && stdin.agent === "vulnerability-scanner") {
    if (!stdin.scan_scope) {
      throw new Error("vulnerability-scanner requires scan_scope");
    }
  }

  return true;
}
```

---

8. Stdout Extraction Pipeline

After an agent completes, `gogent-team-run` extracts and validates its output. The system uses a **three-tier extraction** approach to handle varying output formats.

Tier 1: Direct JSON Parse

When: Entire agent response is valid JSON.

Process:

```
var result map[string]interface{}
if err := json.Unmarshal([]byte(agentOutput), &result); err == nil {
    // Success: agent produced pure JSON
    return result, nil
}
```

Example agent output:

```
{
  "$schema": "security-audit-vulnerability-scanner",
  "status": "complete",
  "metadata": {
    "thinking_budget_used": 4200,
    "files_scanned": 47
  },
  "findings": [...]
}
```

Validation: - Check for required fields: `$schema`, `status`, `metadata.thinking_budget_used` - Warn if missing (logged but non-blocking) - Write to `stdout_{member_id}.json`

Tier 2: Code Block Extraction

When: Agent wraps JSON in markdown code block.

Process:

```go
// Look for ```json ... ``` blocks
re := regexp.MustCompile("```json\n(.*?)\n```")
matches := re.FindStringSubmatch(agentOutput)
if len(matches) > 1 {
    var result map[string]interface{}
    if err := json.Unmarshal([]byte(matches[1]), &result); err == nil {
        // Success: extracted JSON from code block
        return result, nil
    }
}
```

Example agent output:

```
The vulnerability scan found 3 critical issues:

```json
{
  "$schema": "security-audit-vulnerability-scanner",
  "status": "complete",
  "findings": [...]
}
```
```

All critical issues should be addressed before deployment.

```
**Validation:** Same as Tier 1.

### Tier 3: Fallback Envelope

**When:** No valid JSON found in output.

**Process:**
```go
fallback := map[string]interface{}{
    "$schema": resolvedSchema,  // From schema resolution
    "status": "partial",
    "metadata": map[string]interface{}{
        "thinking_budget_used": 0,
    },
    "raw_output": agentOutput,
    "extraction_tier": "fallback",
}
return fallback, nil
```
```

Example agent output:

```
I scanned the codebase and found several SQL injection vulnerabilities
in the authentication module. The main issue is direct string concatenation
in queries. Recommend using parameterized queries instead.
```

Fallback envelope:

```
{
  "$schema": "security-audit-vulnerability-scanner",
  "status": "partial",
  "metadata": {
    "thinking_budget_used": 0
```

```
  },
  "raw_output": "I scanned the codebase and found several SQL injection vulnerabilities...",
  "extraction_tier": "fallback"
}
```

Purpose: System continues gracefully even with non-compliant output. User can inspect `raw_output` field.

Validation Rules

After extraction, validate against contract schema:

```go
func validateStdout(stdout map[string]interface{}, contractPath string) []string {
    warnings := []string{}

    // Required fields
    if _, ok := stdout["$schema"]; !ok {
        warnings = append(warnings, "missing $schema field")
    }

    if status, ok := stdout["status"].(string); !ok {
        warnings = append(warnings, "missing or invalid status field")
    } else if status != "complete" && status != "partial" && status != "failed" {
        warnings = append(warnings, fmt.Sprintf("invalid status value: %s", status))
    }

    if metadata, ok := stdout["metadata"].(map[string]interface{}); !ok {
        warnings = append(warnings, "missing metadata object")
    } else {
        if _, ok := metadata["thinking_budget_used"].(float64); !ok {
            warnings = append(warnings, "missing metadata.thinking_budget_used")
        }
    }

    // Log warnings but don't block
    for _, warning := range warnings {
        log.Printf("WARNING [%s]: %s", contractPath, warning)
    }

    return warnings
}
```

Philosophy: Validation is best-effort. Warnings are logged for debugging but don't prevent extraction. This prevents system lockup from minor contract violations.

Output File Structure

Final stdout file written to team directory:

`/path/to/team-dir/stdout_{member_id}.json`

Contents: - Tier 1/2: Extracted JSON (validated) - Tier 3: Fallback envelope with raw output

Usage: - Wave 2 agents can read these files via Read tool - Inter-wave scripts parse them for synthesis - User views them via `/team-result` command - Monitoring tools parse for status tracking

---

9. Testing Your Skill

Before deploying to production, validate your skill end-to-end.

9.1 Dry Run (Config Generation Only)

Add `--dry-run` flag to your skill:

```
if (args.includes("--dry-run")) {
  // Generate config and stdin files but don't launch
  const teamDir = setupTeamDirectory();
  generateConfig(teamDir);
  generateStdinFiles(teamDir);

  return `
[security-audit] Dry run complete

**Team directory:** ${teamDir}

**Files generated:**
  • config.json (team configuration)
  • stdin_vulnerability-scanner.json
  • stdin_dependency-checker.json
  • stdin_secret-detector.json
  • stdin_security-analyst.json

**Inspect files:**
  cd ${teamDir}
  cat config.json | jq '.waves'
  cat stdin_vulnerability-scanner.json | jq '.'

**Launch manually:**
  gogent-team-run ${teamDir}
  `;
}
```

Verify:

```
# List files
ls -la /path/to/team-dir
# Expect: config.json, stdin_*.json (4 files in our example)

# Check wave count
cat config.json | jq '.waves | length'
# Expect: 2

# Check Wave 1 member count
cat config.json | jq '.waves[0].members | length'
# Expect: 3

# Validate stdin structure
cat stdin_vulnerability-scanner.json | jq '.'
# Verify: agent, workflow, context, scan_scope all present
```

```
# Check budget calculation
cat config.json | jq '{budget_max_usd, budget_remaining_usd, warning_threshold_usd}'
# Verify: Values match template

# Verify member defaults
cat config.json | jq '.waves[0].members[0] | {status, process_pid, cost_actual_usd}'
# Expect: {"status": "pending", "process_pid": null, "cost_actual_usd": 0.0}
```

9.2 Manual Launch

Run `gogent-team-run` directly from terminal:

```
gogent-team-run /path/to/team-dir
```

Monitor in real-time:

```
# Watch runner log
tail -f /path/to/team-dir/runner.log

# Check config for status updates
watch -n 2 'cat /path/to/team-dir/config.json | jq ".waves[].members[] | {agent, status, cost_a

# Monitor heartbeat (updated every 10s)
watch -n 1 'cat /path/to/team-dir/heartbeat'
```

Expected log output:

```
[gogent-team-run] Starting team execution
[gogent-team-run] Team: security-audit-1707412345
[gogent-team-run] Workflow: security-audit
[gogent-team-run] Budget: $3.00

[Wave 1] Starting wave 1 (3 members)
[Wave 1] Spawning: vulnerability-scanner (haiku, $0.10)
[Wave 1] Spawning: dependency-checker (haiku, $0.10)
[Wave 1] Spawning: secret-detector (haiku, $0.10)
[Wave 1] Budget reserved: $0.30, remaining: $2.70

[vulnerability-scanner] PID 42731, started 2026-02-08T14:30:15Z
[dependency-checker] PID 42732, started 2026-02-08T14:30:15Z
[secret-detector] PID 42733, started 2026-02-08T14:30:15Z

[vulnerability-scanner] Completed in 28s, exit 0
[vulnerability-scanner] Cost: $0.07 (estimated $0.10, returned $0.03)
[dependency-checker] Completed in 31s, exit 0
[dependency-checker] Cost: $0.08 (estimated $0.10, returned $0.02)
[secret-detector] Completed in 25s, exit 0
[secret-detector] Cost: $0.06 (estimated $0.10, returned $0.04)

[Wave 1] All members complete
[Wave 1] Running inter-wave script: gogent-team-prepare-synthesis

[gogent-team-prepare-synthesis] Merging 3 stdout files
[gogent-team-prepare-synthesis] Wrote: pre-synthesis.md (14KB)
```

```
[Wave 2] Starting wave 2 (1 member)
[Wave 2] Spawning: security-analyst (sonnet, $1.50)
[Wave 2] Budget reserved: $1.50, remaining: $1.29

[security-analyst] PID 42801, started 2026-02-08T14:31:02Z
[security-analyst] Completed in 55s, exit 0
[security-analyst] Cost: $0.42 (estimated $1.50, returned $1.08)

[Wave 2] All members complete

[gogent-team-run] Team complete in 90s
[gogent-team-run] Total cost: $0.63 / $3.00 (21% used)
[gogent-team-run] Budget remaining: $2.37
```

9.3 Verify Stdout Contracts

After completion, validate all stdout files:

```
cd /path/to/team-dir

# Check all stdout files have $schema field
for f in stdout_*.json; do
  echo "═══ $f ═══"
  jq '."$schema"' "$f"
done

# Expected output:
# ═══ stdout_vulnerability-scanner.json ═══
# "security-audit-vulnerability-scanner"
# ═══ stdout_dependency-checker.json ═══
# "security-audit-dependency-checker"
# ═══ stdout_secret-detector.json ═══
# "security-audit-secret-detector"
# ═══ stdout_security-analyst.json ═══
# "security-audit-analyst"
```

Validate against schemas (if you have `ajv-cli` installed):

```
for f in stdout_*.json; do
  schema=$(jq -r '."$schema"' "$f")
  echo "Validating $f against ${schema}.json"
  ajv validate \
    -s ~/.claude/schemas/teams/stdin-stdout/${schema}.json \
    -d "$f"
done
```

Check for fallback tier (indicates agent didn't follow contract):

```
for f in stdout_*.json; do
  tier=$(jq -r '.extraction_tier // "tier-1-or-2"' "$f")
  if [ "$tier" = "fallback" ]; then
    echo "WARNING: $f used fallback extraction"
  fi
done
```

9.4 Cost Verification

Compare actual vs. estimated costs:

```
cat config.json | jq '
  .waves[].members[] |
  select(.status == "complete") |
  {
    agent,
    estimated: .cost_estimated_usd,
    actual: .cost_actual_usd,
    diff: (.cost_actual_usd - .cost_estimated_usd),
    percent: ((.cost_actual_usd / .cost_estimated_usd) * 100 | round)
  }
'
```

Expected output:

```
{
  "agent": "vulnerability-scanner",
  "estimated": 0.1,
  "actual": 0.07,
  "diff": -0.03,
  "percent": 70
}
{
  "agent": "dependency-checker",
  "estimated": 0.1,
  "actual": 0.08,
  "diff": -0.02,
  "percent": 80
}
{
  "agent": "secret-detector",
  "estimated": 0.1,
  "actual": 0.06,
  "diff": -0.04,
  "percent": 60
}
{
  "agent": "security-analyst",
  "estimated": 1.5,
  "actual": 0.42,
  "diff": -1.08,
  "percent": 28
}
```

Analysis:

- If actual consistently < 70% of estimated -> reduce estimates in template (saves budget)
- If actual consistently > 120% of estimated -> increase estimates (prevents overruns)
- If any agent hits `cost_status: "over_budget"` -> investigate (codebase larger than expected? Agent inefficient?)

9.5 Integration Test

Create a test script that runs the full workflow:

```bash
#!/usr/bin/env bash
set -euo pipefail

SKILL_NAME="security-audit"
TEST_PROJECT="/tmp/test-project"
TEST_OUTPUT="/tmp/test-output"

echo "═══ Integration Test: ${SKILL_NAME} ═══"

# 1. Setup test project
mkdir -p "$TEST_PROJECT"
cat > "$TEST_PROJECT/server.go" <<'EOF'
package main
import "fmt"
func main() {
    apiKey := "sk-1234567890abcdef"  // Hardcoded secret
    fmt.Println("Server starting with key:", apiKey)
}
EOF

# 2. Invoke skill
cd "$TEST_PROJECT"
claude-code --command "/${SKILL_NAME} --dry-run" > "$TEST_OUTPUT/skill-output.txt"

# 3. Verify team directory created
TEAM_DIR=$(grep "Team directory:" "$TEST_OUTPUT/skill-output.txt" | awk '{print $NF}')
if [ ! -d "$TEAM_DIR" ]; then
  echo "FAIL: Team directory not created"
  exit 1
fi

# 4. Verify config structure
jq -e '.workflow_type == "security-audit"' "$TEAM_DIR/config.json" || {
  echo "FAIL: Invalid workflow_type"
  exit 1
}

jq -e '.waves | length == 2' "$TEAM_DIR/config.json" || {
  echo "FAIL: Expected 2 waves"
  exit 1
}

# 5. Verify stdin files
for agent in vulnerability-scanner dependency-checker secret-detector security-analyst; do
  stdin="$TEAM_DIR/stdin_${agent}.json"
  if [ ! -f "$stdin" ]; then
    echo "FAIL: Missing stdin file for $agent"
    exit 1
  fi

  jq -e '.agent and .workflow and .context' "$stdin" || {
```

```
    echo "FAIL: Invalid stdin structure for $agent"
    exit 1
  }
done

# 6. Launch team-run
echo "Launching team-run..."
gogent-team-run "$TEAM_DIR" > "$TEST_OUTPUT/team-run.log" 2>&1 &
TEAM_RUN_PID=$!

# 7. Wait for completion (timeout 5 minutes)
timeout 300 bash -c "while [ ! -f '$TEAM_DIR/runner.log' ] || ! grep -q 'Team complete' '$TEAM_
  echo "FAIL: Team execution timeout"
  kill $TEAM_RUN_PID 2>/dev/null || true
  exit 1
}

# 8. Verify stdout files
for agent in vulnerability-scanner dependency-checker secret-detector security-analyst; do
  stdout="$TEAM_DIR/stdout_${agent}.json"
  if [ ! -f "$stdout" ]; then
    echo "FAIL: Missing stdout file for $agent"
    exit 1
  fi

  jq -e '."$schema" and .status and .metadata.thinking_budget_used' "$stdout" || {
    echo "FAIL: Invalid stdout structure for $agent"
    exit 1
  }
done

# 9. Verify findings (should detect hardcoded API key)
if ! jq -e '.secrets | length > 0' "$TEAM_DIR/stdout_secret-detector.json" >/dev/null; then
  echo "FAIL: secret-detector should have found hardcoded API key"
  exit 1
fi

# 10. Verify synthesis
if [ ! -f "$TEAM_DIR/pre-synthesis.md" ]; then
  echo "FAIL: Missing pre-synthesis.md from inter-wave script"
  exit 1
fi

if ! jq -e '.report.prioritized_findings | length > 0' "$TEAM_DIR/stdout_security-analyst.json"
  echo "FAIL: security-analyst should have produced prioritized findings"
  exit 1
fi

# 11. Verify cost tracking
total_cost=$(jq '[.waves[].members[].cost_actual_usd] | add' "$TEAM_DIR/config.json")
if [ "$(echo "$total_cost > 0" | bc)" != "1" ]; then
  echo "FAIL: No cost recorded"
  exit 1
```

```
fi

echo "═══ Integration Test: PASS ═══"
echo "Total cost: \$${total_cost}"
echo "Output: $TEST_OUTPUT"
echo "Team dir: $TEAM_DIR"
```

Run it:

```
chmod +x test-security-audit.sh
./test-security-audit.sh
```

---

10. Reference: Existing Workflows

Learn from production-validated workflows.

Overview Table

Workflow	Waves	Models	Budget	Actual Cost	Runtime	Inter-Wave Script	Validation Date
review	1	3 haiku1 sonnet	$2.00	$0.068	38s	None	2026-02-08 (TC-013a)
braintrust	2	3 opus	$16.00	$2.48	6.5min	gogent-team-prepare-synthesis	2026-02-08 (TC-013b)
implementation	N (DAG)	N sonnet	$10.00	$0.86	~5min	None	2026-02-08 (TC-013c)

Review Workflow

Purpose: Parallel code review across multiple domains.

Wave structure:

```
Wave 1 (parallel):
  • backend-reviewer (haiku)
  • frontend-reviewer (haiku)
  • standards-reviewer (haiku)
  • orchestrator (sonnet) — synthesizes findings
```

Key characteristics:

- Single wave (fastest possible)
- No inter-wave processing
- Reviewers produce independent findings
- Orchestrator synthesizes in same wave (reads other outputs via Read tool)

Contract schemas:

- `schemas/teams/stdin-stdout/review-backend-reviewer.json`
- `schemas/teams/stdin-stdout/review-frontend-reviewer.json`
- `schemas/teams/stdin-stdout/review-standards-reviewer.json`
- `schemas/teams/stdin-stdout/review-orchestrator.json`

Cost breakdown (TC-013a):

Agent	Estimated	Actual	Efficiency
backend-reviewer	$0.03	$0.021	70%
frontend-reviewer	$0.03	$0.019	63%
standards-reviewer	$0.03	$0.018	60%
orchestrator	$1.50	$0.010	0.7%

Total: $0.068 (3.4% of budget)

Files:

- Template: `~/.claude/schemas/teams/review.json`
- Skill: `~/.claude/skills/review/SKILL.md`
- Validation: `tickets/team-coordination/tickets/TC-013a.md`

Braintrust Workflow

Purpose: Deep multi-perspective analysis with synthesis.

Wave structure:

```
Wave 1 (parallel):
  • einstein (opus) — Theoretical analysis
  • staff-architect (opus) — Critical review
    ↓
    on_complete_script: gogent-team-prepare-synthesis
    Creates: pre-synthesis.md (45KB)
    ↓
Wave 2 (sequential):
  • beethoven (opus) — Unified synthesis document
```

Key characteristics:

- Two waves with inter-wave synthesis
- Wave 1 produces orthogonal analyses (no overlap)
- Inter-wave script merges analyses into pre-synthesis.md
- Beethoven reads pre-synthesis.md at runtime (file doesn't exist when stdin is written)

Contract schemas:

- `schemas/teams/stdin-stdout/braintrust-einstein.json`
- `schemas/teams/stdin-stdout/braintrust-staff-architect.json`
- `schemas/teams/stdin-stdout/braintrust-beethoven.json`

Cost breakdown (TC-013b):

Agent	Estimated	Actual	Efficiency
einstein	$5.00	$0.95	19%
staff-architect	$5.00	$1.13	23%
beethoven	$5.00	$0.40	8%

Total: $2.48 (15.5% of budget)

Wave timing:

- Wave 1: 3.7 minutes (parallel)
- Inter-wave: 2 seconds (synthesis script)
- Wave 2: 2.8 minutes
- Total: 6.5 minutes

Files:

- Template: `~/.claude/schemas/teams/braintrust.json`
- Skill: `~/.claude/skills/braintrust/SKILL.md`
- Synthesis script: `cmd/gogent-team-prepare-synthesis/main.go`
- Validation: `tickets/team-coordination/tickets/TC-013b.md`

Implementation Workflow

Purpose: DAG-ordered parallel implementation with dependency management.

Wave structure (computed dynamically via Kahn's algorithm):

```
Example from TC-013c:

Wave 1 (parallel):
  • task-001 (go-pro, sonnet) — Implement main.go
  • task-002 (go-pro, sonnet) — Implement main_test.go (blocked_by: task-001)

Wave 2:
  (task-002 runs here due to blocked_by dependency)
```

Key characteristics:

- N waves computed from task DAG
- Uses `gogent-plan-impl` to convert implementation-plan.json -> team config
- Agents create source files using Write/Edit tools
- `augmentToolsForImplementation()` adds Write/Edit to allowed_tools at runtime
- Failure propagation: If task-001 fails -> task-002 skipped

Contract schemas:

- `schemas/teams/stdin-stdout/implementation-worker.json` (generic for all go-pro agents)

Cost breakdown (TC-013c):

Task	Agent	Estimated	Actual	Efficiency
task-001	go-pro	$1.50	$0.45	30%
task-002	go-pro	$1.50	$0.41	27%

Total: $0.86 (8.6% of budget)

Files:

- Template: `~/.claude/schemas/teams/implementation.json`
- Skill: `~/.claude/skills/implement/SKILL.md`
- Plan converter: `cmd/gogent-plan-impl/main.go`
- Validation: `tickets/team-coordination/tickets/TC-013c.md`

Critical bug fix (TC-013c):

Original issue: `go-pro` has `allowed_tools: ["Read", "Glob", "Grep", "Bash"]` in agents-index.json (no Write/Edit).

Impact: Implementation workers couldn't create source files.

Fix: `augmentToolsForImplementation()` in `internal/spawn/spawn.go` adds Write+Edit when `workflowType == "implementation"`.

```go
func augmentToolsForImplementation(agent Agent, workflowType string) Agent {
    if workflowType == "implementation" {
        tools := agent.CLIFlags.AllowedTools
        if !contains(tools, "Write") {
            tools = append(tools, "Write")
        }
        if !contains(tools, "Edit") {
            tools = append(tools, "Edit")
        }
        agent.CLIFlags.AllowedTools = tools
    }
    return agent
}
```

Lesson: Don't modify read-only `allowed_tools` in agents-index.json for workflow-specific needs. Use runtime augmentation.

---

11. Reference: Complete File Inventory

All files involved in the team orchestration system.

Go Binaries (Execution Layer)

Binary	Location	Purpose	Used By
`gogent-team-run`	`cmd/gogent-team-run/`	Main orchestrator daemon (spawns agents, tracks costs, manages waves)	All team workflows
`gogent-team-prepare-synthesis`	`cmd/gogent-team-prepare-synthesis/`	Inter-wave synthesis (merges Wave N stdout into pre-synthesis.md)	braintrust, security-audit
`gogent-plan-impl`	`cmd/gogent-plan-impl/`	Converts implementation-plan.json -> team config + stdin files	implementation

Schemas (Contract Layer)

Master Schemas

Schema	Location	Purpose
`team-config.json`	`schemas/teams/`	Master schema for config.json structure (waves, members, budget)
`common-envelope.json`	`schemas/teams/`	Common stdin fields required for all agents

Workflow Templates

Template	Location	Defines
`review.json`	`schemas/teams/`	Review workflow (4 agents, 1 wave)
`braintrust.json`	`schemas/teams/`	Braintrust workflow (3 agents, 2 waves)
`implementation.json`	`schemas/teams/`	Implementation workflow (N agents, N waves)

Stdin/Stdout Contracts

Contract	Location	Workflow	Agent
`review-backend-reviewer.json`	`schemas/teams/stdin-stdout/`	review	backend-reviewer
`review-frontend-reviewer.json`	`schemas/teams/stdin-stdout/`	review	frontend-reviewer
`review-standards-reviewer.json`	`schemas/teams/stdin-stdout/`	review	standards-reviewer
`review-orchestrator.json`	`schemas/teams/stdin-stdout/`	review	orchestrator
`braintrust-einstein.json`	`schemas/teams/stdin-stdout/`	braintrust	einstein
`braintrust-staff-architect.json`	`schemas/teams/stdin-stdout/`	braintrust	staff-architect
`braintrust-beethoven.json`	`schemas/teams/stdin-stdout/`	braintrust	beethoven
`implementation-worker.json`	`schemas/teams/stdin-stdout/`	implementation	go-pro (generic)

Configuration Files

File	Location	Purpose
`agents-index.json`	`.claude/agents/`	Agent registry (model, tier, tools, relationships)
`settings.json`	`.claude/`	Session settings (use_team_pattern flag)

Runtime Files (Per Team Execution)

Created in: `$GOGENT_SESSION_DIR/teams/{timestamp}.{workflow_name}/`

File	Purpose	Written By	Read By
`config.json`	Team state (waves, members, status, budget)	Skill -> gogent-team-run	gogent-team-run, monitoring tools
`stdin_{member_id}.jsonl`	Agent input (N files, one per member)	Skill or gogent-plan-impl	claude CLI (via stdin pipe)

File	Purpose	Written By	Read By
`stdout_{member_id}.jsonl`	Agent output (N files, one per member)	claude CLI (via stdout capture)	gogent-team-run, Wave 2 agents, user
`runner.log`	Execution log (verbose)	gogent-team-run	User (for debugging)
`heartbeat`	Health monitoring (updated every 10s)	gogent-team-run	Monitoring tools
`gogent-team-run.pid`	PID file (for signal handling)	gogent-team-run	User (for manual kill)
`pre-synthesis.md`	Inter-wave artifact (Wave 1 -> Wave 2)	gogent-team-prepare-synthesis	Wave 2 agents (via Read tool)

Skills (User Interface Layer)

Skill	Location	Launches
`/review`	`~/.claude/skills/review/SKILL.md`	review workflow
`/braintrust`	`~/.claude/skills/braintrust/SKILL.md`	braintrust workflow
`/implement`	`~/.claude/skills/implement/SKILL.md`	implementation workflow
`/team-status`	`~/.claude/skills/team-status/SKILL.md`	Shows real-time team progress
`/team-result`	`~/.claude/skills/team-result/SKILL.md`	Displays final stdout files
`/team-cancel`	`~/.claude/skills/team-cancel/SKILL.md`	Graceful team shutdown

Validation Tickets (Documentation)

Ticket	Location	Validates
TC-013a	`tickets/team-coordination/tickets/TC-013a.md`	review workflow end-to-end
TC-013b	`tickets/team-coordination/tickets/TC-013b.md`	braintrust workflow end-to-end
TC-013c	`tickets/team-coordination/tickets/TC-013c.md`	implementation workflow end-to-end + augmentToolsForImplementation fix

Architecture Documentation

Document	Location	Covers
`PARALLEL-ORCHESTRATION-DESIGN.md`	Project root	Original design doc (phase 3A/3B patterns)
`TEAM-RUN-FRAMEWORK.md`	`docs/`	Commercial extensibility guide
`SKILL-AUTHORING-GUIDE.md`	`docs/teams/`	This document

---

12. Checklist: New Team Skill

Use this checklist when creating a new team composition skill.

Planning Phase

- ☐ Workflow purpose defined (one sentence)
- ☐ Agents identified (roles, responsibilities)
- ☐ Wave structure designed (parallel, sequential, DAG)
- ☐ Budget estimated (model costs + 50% margin)
- ☐ Inter-wave processing needs identified (if multi-wave)

Agent Registration

- ☐ All agents added to `~/.claude/agents/agents-index.json`
- ☐ `model` field set (haiku/sonnet/opus)
- ☐ `tier` field set (1/2/3)
- ☐ `cli_flags.allowed_tools` includes needed tools
- ☐ `spawned_by` includes skill or router
- ☐ `can_spawn` set if agent spawns children

Schema Creation

- ☐ Stdin/stdout contract files created in `~/.claude/schemas/teams/stdin-stdout/`
- ☐ Each contract has both `stdin` and `stdout` sections
- ☐ Stdin includes common envelope fields (agent, workflow, context, description)
- ☐ Stdout includes required fields ($schema, status, metadata.thinking_budget_used)
- ☐ Workflow-specific fields documented in contracts
- ☐ Filenames follow convention: `{workflow_type}-{agent_base}.json`

Template Creation

- ☐ Team config template created in `~/.claude/schemas/teams/{workflow}.json`
- ☐ Template references `team-config.json` schema
- ☐ `workflow_type` string chosen and used consistently
- ☐ All waves defined with correct member structures
- ☐ Member defaults correct (status: pending, pids: null, costs: 0.0)
- ☐ Budget fields calculated (max, remaining, warning_threshold)
- ☐ Timeouts appropriate for models (haiku: 120s, sonnet: 300s, opus: 600s)
- ☐ `blocked_by` and `blocks` set correctly (if DAG-ordered)

Skill Implementation

- ☐ SKILL.md created in `~/.claude/skills/{skill_name}/`
- ☐ YAML frontmatter includes name, description, version, triggers
- ☐ Two dispatch paths implemented (team-run default, foreground fallback)
- ☐ Team directory creation logic correct
- ☐ Template population fills all dynamic fields
- ☐ Stdin generation creates file for each member
- ☐ Stdin files include common envelope + workflow-specific fields
- ☐ Config.json write happens after stdin files
- ☐ Background launch uses `gogent-team-run "$team_dir" &`
- ☐ Launch verification checks for `background_pid` in config
- ☐ User-facing output includes /team-status, /team-result, /team-cancel instructions

Inter-Wave Script (if applicable)

- ☐ Script takes team directory as first argument
- ☐ Script reads stdout files from previous wave
- ☐ Script writes artifact to team directory
- ☐ Script exits 0 on success, non-zero on failure
- ☐ Script logs to stderr (captured in runner.log)
- ☐ Script referenced in `waves[N].on_complete_script` in template
- ☐ Wave N+1 stdin references artifact path (will be created before Wave N+1 runs)

Testing

- ☐ Dry run flag implemented (generates config without launching)
- ☐ Dry run verified: config.json + stdin files correct
- ☐ Manual launch tested: `gogent-team-run /path/to/team-dir`
- ☐ Runner log monitored: all agents spawn and complete
- ☐ Stdout files validated: all have `$schema` and `status` fields
- ☐ Cost verification: actual vs estimated compared
- ☐ Budget tracking: remaining budget >= 0 after completion
- ☐ Integration test created (script that runs end-to-end)
- ☐ Edge cases tested: agent failure, timeout, budget overrun

Documentation

- ☐ Workflow added to this guide's "Existing Workflows" section
- ☐ Cost estimates updated based on actual production data
- ☐ Common pitfalls documented (if any discovered during testing)
- ☐ Validation ticket created (TC-XXX in tickets/team-coordination/)

Deployment

- ☐ Skill symlinked or copied to `~/.claude/skills/`
- ☐ Inter-wave script compiled and installed to `$PATH`
- ☐ Agents available in agents-index.json
- ☐ Settings.json has `use_team_pattern: true`
- ☐ Smoke test run on real project
- ☐ User documentation updated (README, skill catalog)

Appendix A: Common Patterns

Pattern: User-Scoped Configuration

Allow users to customize workflow via flags:

```
// Parse user flags
const scope = args.includes("--scope") ? args[args.indexOf("--scope") + 1] : "all";
const exclude = args.includes("--exclude") ? args[args.indexOf("--exclude") + 1].split(",") :
const focus = args.includes("--focus") ? args[args.indexOf("--focus") + 1].split(",") : [];

// Apply to stdin generation
const stdinReviewer = {
  agent: "backend-reviewer",
  workflow: "review",
```

```
  context: { ... },
  review_scope: {
    file_patterns: scope === "backend" ? ["**/*.go"] : ["**/*"],
    exclude_patterns: exclude.concat(["**/node_modules/**"]),
  },
  focus_areas: focus.length > 0 ? focus : ["correctness", "performance"],
};
```

Pattern: Context Injection

Automatically inject relevant project context:

```
// Detect project type
const projectType = (() => {
  if (fs.existsSync("go.mod")) return "go";
  if (fs.existsSync("package.json")) return "javascript";
  if (fs.existsSync("requirements.txt")) return "python";
  return "unknown";
})();

// Load conventions
const conventions = {
  convention_files: projectType === "go" ? ["go.md", "go-api.md"] : [],
};

// Add to stdin
stdin.project_conventions = conventions;
```

Pattern: Progressive Budget Increase

Start with conservative budget, increase if hit:

```
const baseBudget = 3.0;
let currentBudget = baseBudget;

// After first attempt
if (teamStatus === "over_budget") {
  currentBudget = baseBudget * 2;
  console.log(`Budget exhausted. Retrying with $${currentBudget.toFixed(2)}`);
  // Regenerate config with new budget and relaunch
}
```

Pattern: Conditional Wave Inclusion

Enable/disable waves based on project characteristics:

```
// Optional security wave for production code
const waves = [baseWaves];

if (isProductionCode(projectRoot)) {
  waves.push({
    wave_number: waves.length + 1,
    description: "Security audit",
    members: [
```

```
      {agent: "security-reviewer", model: "sonnet", ...}
    ]
  });
}

template.waves = waves;
```

Appendix B: Debugging Techniques

Technique 1: Verbose Runner Logging

Set GOGENT_DEBUG=1 before launching:

```
GOGENT_DEBUG=1 gogent-team-run /path/to/team-dir
```

Produces detailed logs:

```
[DEBUG] Reading config from /path/to/team-dir/config.json
[DEBUG] Workflow: security-audit
[DEBUG] Waves: 2
[DEBUG] Budget: $3.00
[DEBUG] Wave 1: 3 members
[DEBUG] Resolving stdout schema for workflow=security-audit agent=vulnerability-scanner
[DEBUG]   Candidate 1: security-audit-vulnerability-scanner.json -> FOUND
[DEBUG] Spawning: claude --agent vulnerability-scanner < stdin_vulnerability-scanner.json
[DEBUG] PID: 42731
[DEBUG] Monitoring process 42731 (timeout: 120s)
[DEBUG] Process 42731 exited: code=0
[DEBUG] Extracting stdout from process 42731
[DEBUG] Tier 1 (direct JSON): SUCCESS
[DEBUG] Validating stdout against security-audit-vulnerability-scanner.json
[DEBUG]   Required fields: OK
[DEBUG]   Status value: complete -> OK
[DEBUG] Reconciling cost: estimated=$0.10 actual=$0.07 diff=-$0.03
[DEBUG] Budget remaining: $2.93
```

Technique 2: Inspect Intermediate Files

Read stdin files before launch to verify structure:

```
cat /path/to/team-dir/stdin_vulnerability-scanner.json | jq '.'
```

Read stdout files after completion to see raw agent output:

```
cat /path/to/team-dir/stdout_vulnerability-scanner.json | jq '.'
```

Check for extraction tier (fallback = problem):

```
jq '.extraction_tier // "tier-1-or-2"' /path/to/team-dir/stdout_*.json
```

Technique 3: Manual Agent Invocation

Spawn an agent manually to test stdin contract:

```
cat /path/to/team-dir/stdin_vulnerability-scanner.json | \
  claude --agent vulnerability-scanner > /tmp/test-output.json

cat /tmp/test-output.json | jq '.'
```

Technique 4: Cost Trace

Track budget changes wave-by-wave:

```
watch -n 2 'cat /path/to/team-dir/config.json | jq "{
  budget_max: .budget_max_usd,
  budget_remaining: .budget_remaining_usd,
  wave_1_cost: [.waves[0].members[].cost_actual_usd] | add,
  wave_2_cost: [.waves[1].members[].cost_actual_usd] | add
}"'
```

Technique 5: Heartbeat Monitoring

Check daemon health (updated every 10s):

```
watch -n 1 cat /path/to/team-dir/heartbeat
```

If heartbeat stops updating -> daemon crashed (check runner.log for panic).

Appendix C: Performance Tuning

Tuning 1: Adjust Timeouts

If agents consistently hit timeout:

```
{
  "timeout_ms": 300000  // Increase from 120000 for large codebases
}
```

Monitor actual completion times:

```
jq '.waves[].members[] | {agent, started_at, completed_at}' config.json
```

Calculate duration and adjust timeouts to 2x typical duration.

Tuning 2: Parallelize More Aggressively

If agents are independent, merge waves:

Before (sequential):

```
{
  "waves": [
    {"wave_number": 1, "members": [{"agent": "agent-A"}]},
    {"wave_number": 2, "members": [{"agent": "agent-B"}]}
  ]
}
```

After (parallel):

```
{
  "waves": [
    {
      "wave_number": 1,
      "members": [
        {"agent": "agent-A"},
        {"agent": "agent-B"}
      ]
    }
  ]
}
```

Runtime reduced from `time_A + time_B` to `max(time_A, time_B)`.

Tuning 3: Use Haiku for Simple Tasks

If sonnet agents are doing mechanical work, downgrade to haiku:

Before:

```
{"agent": "code-formatter", "model": "sonnet", "cost_estimated_usd": 1.5}
```

After:

```
{"agent": "code-formatter", "model": "haiku", "cost_estimated_usd": 0.1}
```

Cost reduced 15x, typically with no quality loss for mechanical tasks.

Tuning 4: Reduce Thinking Budget

If agents use extended thinking excessively, reduce budget in stdin:

```
{
  "thinking_budget": 2000  // Tokens (default is model-dependent)
}
```

Monitor actual usage:

```
jq '.metadata.thinking_budget_used' /path/to/team-dir/stdout_*.json
```

If typically using <50% of budget, reduce allocation.

This guide provides everything you need to author robust, production-quality team composition skills for GOgent-Fortress. Start with the examples, test thoroughly, and iterate based on actual cost and runtime data. The system is designed for extensibility—your workflow is a first-class citizen alongside `/review`, `/braintrust`, and `/implement`.